

Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

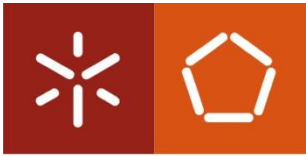
2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães

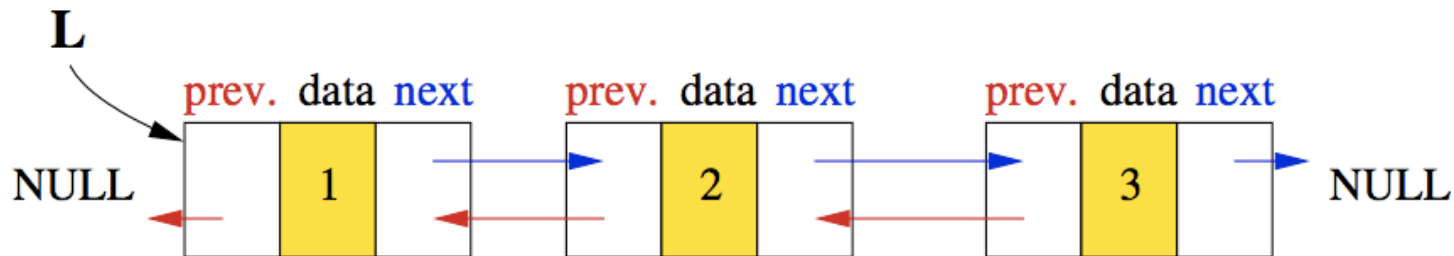


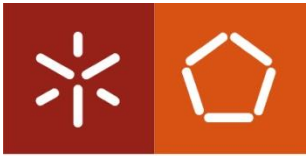
Listas Duplamente Ligadas

Universidade do Minho

Lista Duplamente Ligada (*Double Linked List*):

- Permite uma navegação em frente e reversa (para trás)
- Por exemplo, a remoção torna-se mais intuitiva
- Cada nodo armazena também o nodo anterior (previous);





Listas Duplamente Ligadas

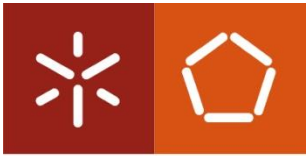
Universidade do Minho

- Exemplo:

```
/* definição do tipo de dados a armazenar em cada nó */  
typedef int data_t;
```

```
/* definição de um nó, contendo os dados, o apontador  
para o próximo elemento e o apontador para o anterior*/
```

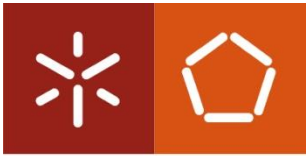
```
typedef struct node {  
    data_t data;  
    struct node* next;  
    struct node* previous;  
} node_t;  
node_t *lista;
```



Listas Duplamente Ligadas

Universidade do Minho

- Operações mais comuns:
 - Inserir elemento
 - Remover elemento
 - Percorrer lista
- Inserir elemento
 - Inserir no início
 - Inserir no fim
 - ... (podem existir outras variantes, por exemplo, inserir de forma ordenada)



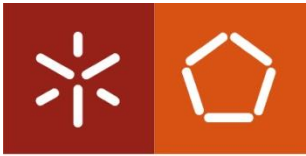
Listas Duplamente Ligadas

Universidade do Minho

- Inserir no início
 - Uma possível implementação em C

```
void inserirInicio(node_t **lista, node_t *elemento){  
    if(*lista == NULL){  
        *lista = elemento;  
    }  
    else{  
        elemento->next = *lista;  
        /*supondo que o campo previous de elemento está a NULL*/  
        (*lista)->previous=elemento;  
        *lista = elemento;  
    }  
}
```

A ordem das operações
é importante!!!

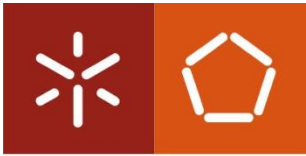


Listas Duplamente Ligadas

Universidade do Minho

- Inserir no fim

```
void inserirFim(node_t **lista, node_t *elemento){
    node_t *aux;
    aux = *lista;
    if(*lista == NULL){
        *lista = elemento;
    }
    else{
        /* Percorre a lista até encontrar o último elemento */
        while (aux->next != NULL){
            aux = aux->next;
        }
        aux->next = elemento;
        elemento->previous=aux;
    }
}
```

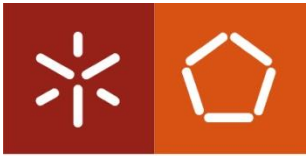


Listas Duplamente Ligadas

Universidade do Minho

- Inserir no fim
 - Se mantivermos um apontador para o último elemento da lista, a inserção torna-se mais simples.

```
void inserirFim(node_t **lista, node_t **fim, node_t *elemento){  
    if(*lista == NULL){  
        *lista = elemento;  
        *fim=elemento;  
    }  
    else{  
        (*fim)->next = elemento;  
        elemento->previous=*fim;  
        *fim=elemento;  
    }  
}
```

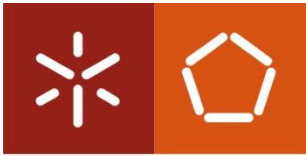


Listas Duplamente Ligadas

Universidade do Minho

- Remover um elemento

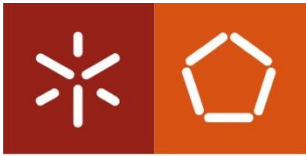
```
int removerElemento(node_t **lista, data_t valorProcurado){
    node_t *aux;
    int encontrou=0;
    aux = *lista;
    /* Percorre a lista até encontrar o elemento (nó) procurado */
    while (encontrou == 0 && aux != NULL){
        if (igual(aux->data,valorProcurado)) /* função igual a implementar */
            encontrou = 1;
        else
            aux=aux->next;
    }
```



Listas Duplamente Ligadas

Universidade do Minho

```
if (encontrou == 1){  
    /* Verifica se o elemento (no') encontrado e' o primeiro da lista */  
    if (aux->previous == NULL && aux == *lista){  
        *lista = aux->next;  
        (*lista)->previous=NULL;  
    }  
    /*Caso o elemento(nó) encontrado nao seja o primeiro da lista*/  
    else{  
        (aux->previous)->next=aux->next;  
        aux->next->previous=aux->previous;  
    }  
    /* Liberta a memoria alocada */  
    free(aux);  
    aux = NULL;  
    return 1;  
}  
/*Retorna zero indicando que o elemento(no) nao foi encontrado*/  
return 0;  
}
```



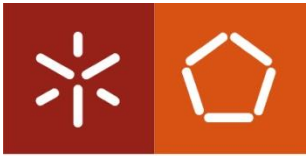
Listas Duplamente Ligadas

Universidade do Minho

- Travessia – $O(n)$:
 - percorrer todos elementos da lista
- Algoritmos (para a frente e para trás):

```
Node *aux = first;
while (aux!= NULL)
{
    // processamento
    aux = aux->next;
}
```

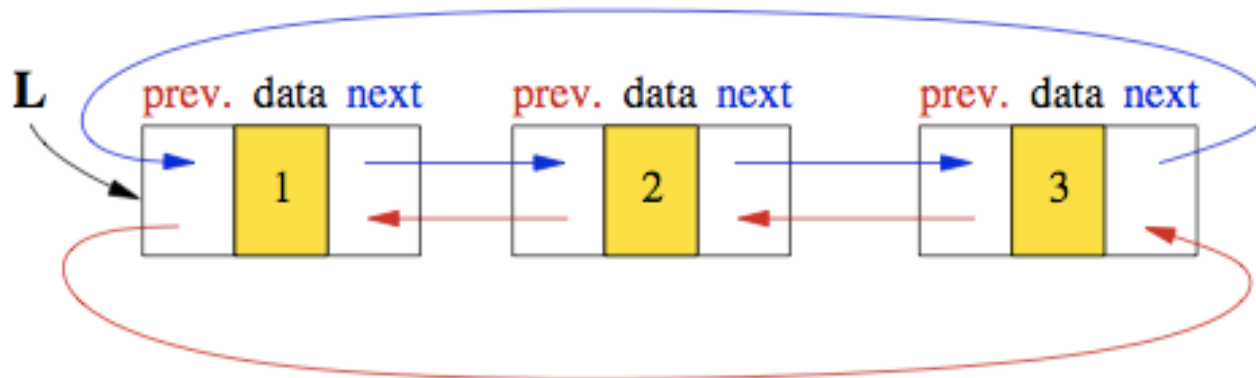
```
Node *aux = last;
while (aux!= NULL)
{
    // processamento
    aux = aux->previous;
}
```

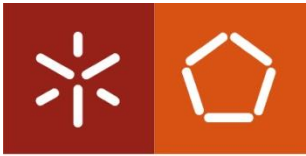


Listas Circulares

Universidade do Minho

- O último nodo liga-se ao primeiro, deixa de haver ligações 0 (NULL)
- A rigor, neste caso, não faz sentido falarmos em primeiro ou último elemento.
- A lista pode ser representada por um apontador para um elemento inicial qualquer da lista.
- Permite uma navegação completa a partir de qualquer nodo
- Pode haver listas circulares simples e duplamente ligadas
- Implementação
 - Em termos das estruturas de dados não existem diferenças para as listas não circulares

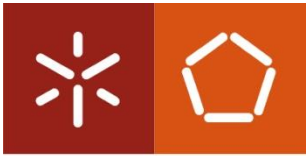




Universidade do Minho

Listas Circulares

- Operações mais comuns:
 - Inserir elemento
 - Remover elemento
 - Percorrer lista
- Inserir elemento
 - Inserir no início
 - Inserir no fim
 - ... (podem existir outras variantes, por exemplo, inserir de forma ordenada)



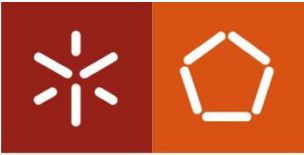
Universidade do Minho

Listas Circulares

- Inserir no início
 - Uma possível implementação em C

```
void inserirInicio(node_t **lista, node_t *elemento){  
    if(*lista == NULL){  
        *lista = elemento;  
        elemento->next=*lista;  
        elemento->previous=*lista;  
    }  
    else{  
        elemento->next =*lista;  
        elemento->previous=(*lista)->previous;  
        (*lista)->previous->next=elemento;  
        (*lista)->previous=elemento;  
        *lista = elemento;  
    }  
}
```

A ordem das operações
é importante!!!



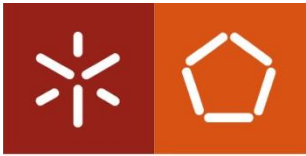
Universidade do Minho

Listas Circulares

- Inserir no fim

```
void inserirFim(node_t **lista, node_t *elemento){  
    node_t *aux;  
    aux = *lista;  
    if(*lista == NULL){  
        *lista = elemento;  
        elemento->next=*lista;  
        elemento->previous=*lista;  
    }  
    else{  
        /* numa lista circular duplamente ligada o previous do primeiro elemento aponta para o último  
        elemento */  
        aux = aux->previous;  
        aux->next = elemento;  
        elemento->previous=aux;  
        elemento->next=*lista;  
        (*lista)->previous=elemento;  
    }  
}
```

A ordem das operações é importante!!!

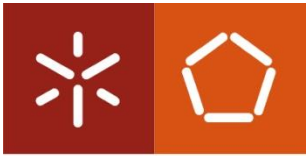


Listas Circulares

Universidade do Minho

- Inserir no fim
 - Se mantivermos um apontador para a último elemento da lista, não há muita diferença porque se trata de uma lista circular.

```
void inserirFim(node_t **lista, node_t **fim, node_t *elemento){  
    if(*lista == NULL){  
        *lista = elemento;  
        elemento->next=*lista;  
        elemento->previous=*lista;  
        *fim=elemento;  
    } else {  
        (*fim)->next = elemento;  
        elemento->previous=*fim;  
        elemento->next=*lista;  
        (*lista)->previous=elemento;  
        *fim=elemento;  
    }  
}
```

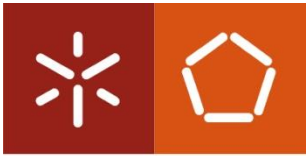


Listas Circulares

Universidade do Minho

- Remover um elemento

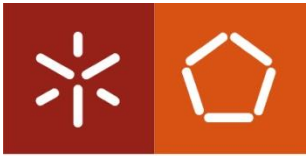
```
int removerElemento(node_t **lista, data_t valorProcurado){
    node_t *aux;
    int encontrou=0;
    aux = *lista;
    /* Percorre a lista até encontrar o elemento (nó) procurado */
    while (encontrou == 0 && aux->next != *lista){
        if (igual(aux->data,valorProcurado)) /* função igual a implementar */
            encontrou = 1;
        else
            aux=aux->next;
    }
```



Listas Circulares

Universidade do Minho

```
if (encontrou == 1){  
    /* Verifica se o elemento (no') encontrado e' o primeiro da lista */  
    if (aux->previous->next == *lista && aux == *lista){  
        *lista = aux->next;  
    }  
    /*Caso o elemento(nó) encontrado nao seja o primeiro da lista*/  
  
    (aux->previous)->next=aux->next;  
    aux->next->previous=aux->previous;  
  
    /* Liberta a memoria alocada */  
    free(aux);  
    aux = NULL;  
    return 1;  
}  
/*Retorna zero indicando que o elemento(no) nao foi encontrado*/  
return 0;  
}
```



Listas Circulares

Universidade do Minho

- Travessia – $O(n)$:
 - percorrer todos elementos da lista
- Algoritmo:
 - Para percorrer os elementos de uma lista circular, visitamos todos os elementos a partir do apontador do elemento inicial (mas na verdade podemos começar em qualquer elemento) até alcançarmos novamente esse mesmo elemento.

```
Node *aux = first; /* pode ser o elemento que quisermos */
if (aux==NULL)
{ /* lista vazia */ }
else {
do
{
    // processamento
    aux = aux->next;
} while (aux!=first);
}
```